LABORATORIO 11

Microservicios con Spring Boot

Desarrollo de Aplicaciones Web

Sección C24-D

Jesús Castillo Sumire

Objetivo

Desarrollar servicios REST independientes para administrar datos de tienda y un sistema de citas médicas, aplicando arquitectura por capas, persistencia JPA/Hibernate, DTO, validación, configuración por ambientes y comunicación HTTP entre servicios.

Cómo se trabajó

La guía incluye un ejercicio de tienda y una actividad final de Salud Integral. Organicé ambos bloques en siete proyectos independientes: cuatro para tienda y tres para clínica. Cada servicio tiene su puerto y base de datos; la colección de Postman contiene los flujos y escenarios de error.

Herramientas

Java 21, Spring Boot 4.1.0, Maven Wrapper, Spring Web MVC, Spring Data JPA, Hibernate, Bean Validation, MariaDB de XAMPP, YAML y Postman.

Flujo y procedimiento

1. Iniciar MySQL y preparar las bases. 2. Ejecutar cada servicio en terminal independiente. 3. Crear primero los registros maestros. 4. Probar altas, consultas, estados y errores desde Postman.

La guía contiene el ejercicio de tienda y luego la actividad clínica; el informe los presenta como bloques separados.

1. Organización y responsabilidades

Cada proyecto contiene controlador, servicio, repositorio, modelo, DTO y configuración YAML. Los IDs que apuntan a datos de otro servicio se verifican mediante HTTP.

Bloque	Servicio	Puerto	Base de datos	Responsabilidad
Tienda	categoria-service	8082	bd_categoria	CRUD de categorías.
Tienda	cliente-service	8081	bd_cliente	CRUD de clientes.
Tienda	producto-service	8083	bd_producto	CRUD de productos; valida categoría por HTTP.
Tienda	pedido-service	8084	bd_pedido	Pedidos y detalles; verifica cliente y productos.
Clínica	paciente-service	8091	bd_paciente	CRUD de pacientes con validaciones.
Clínica	medico-service	8092	bd_medico	Especialidades, médicos y horarios.
Clínica	cita-service	8093	bd_cita	Citas, estados y atenciones; verifica IDs remotos.

Diagrama de arquitectura

Diagrama original del diseño; no es una captura de pantalla de la aplicación.

2. Bases y perfiles YAML

El script SQL crea los esquemas de tienda y clínica. El perfil dev usa bases con sufijo _dev, createDatabaseIfNotExist=true y ddl-auto update. Con permisos para crear bases, Hibernate genera o actualiza las tablas. El perfil prod usa las bases sin sufijo y ddl-auto validate.

application.yml - selección del perfil

PROFILE_ACTIVE permite cambiar el ambiente sin editar Java; dev es el valor predeterminado.

```
spring:
  application:
    name: cita-service
  profiles:
    active: ${PROFILE_ACTIVE:dev}
```

application-dev.yml - XAMPP MariaDB

Las variables permiten configurar puerto y conexión; el dialecto corresponde al MariaDB distribuido con XAMPP.

```
server:
  port: ${SERVER_PORT:8093}
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/
bd_cita_dev?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC}
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:}
  jpa:
    database-platform: org.hibernate.dialect.MariaDBDialect
    hibernate:
      ddl-auto: update
    show-sql: true
```

application-prod.yml - esquema existente

En prod se valida el esquema y no se alteran tablas al iniciar.

```
server:
  port: ${SERVER_PORT:9093}
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/bd_cita?useSSL=false&serverTimezone=UTC}
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:}
  jpa:
    database-platform: org.hibernate.dialect.MariaDBDialect
    hibernate:
      ddl-auto: validate
    show-sql: false
```

3. Entidades, DTO y validación

La entidad representa datos persistidos; el DTO limita el contenido aceptado por la API.

Entidad Cita - campos esenciales

Paciente y médico son IDs remotos, no relaciones JPA con una base externa. Se validan por REST.

```
@Entity
@Table(name = "cita")
public class Cita {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_cita")
    private Long id;

    @NotNull
    @Column(nullable = false)
    private LocalDate fecha;

    @NotNull
    @Column(nullable = false)
    private LocalTime hora;

    @NotNull
    @Column(name = "id_paciente", nullable = false)
    private Long idPaciente;

    @NotNull
    @Column(name = "id_medico", nullable = false)
    private Long idMedico;

    @Column(nullable = false, length = 20)
    private String estado = "PROGRAMADA";
}
```

DTO para crear cita

@Valid aplica estas restricciones antes de llamar al servicio.

```
public record CitaRequest(
    @NotNull LocalDate fecha,
    @NotNull LocalTime hora,
    @NotNull @Positive Long idPaciente,
    @NotNull @Positive Long idMedico,
    @Size(max = 300) String motivo
) {}
```

Atención - relación local uno a uno

La atención y la cita comparten la base bd_cita; la columna única impide duplicar la atención por cita.

```
@OneToOne(optional = false)
@JoinColumn(name = "id_cita", unique = true)
private Cita cita;

@Lob private String diagnostico;
@Lob private String tratamiento;
@Lob private String observaciones;
```

4. Comunicación REST y reglas

Cita valida paciente y médico mediante llamadas HTTP antes de guardar. Pedido valida cliente y productos; producto valida la categoría. Así se evita persistir una referencia que no existe.

Validación remota de referencias

El servicio guarda la cita solo si ambas consultas remotas encuentran sus recursos.

```
        private void validarServicios(Long idPaciente, Long idMedico) {
            pacientes.get().uri("/api/pacientes/{id}", idPaciente)
                .retrieve().toBodilessEntity();
            medicos.get().uri("/api/medicos/{id}", idMedico)
                .retrieve().toBodilessEntity();
        }

        public Cita crear(CitaRequest datos) {
            validarServicios(datos.idPaciente(), datos.idMedico());
            Cita cita = new Cita();
            cita.setFecha(datos.fecha());
            cita.setHora(datos.hora());
            cita.setIdPaciente(datos.idPaciente());
            cita.setIdMedico(datos.idMedico());
            cita.setMotivo(datos.motivo());
            return repository.save(cita);
        }
```

Estados de cita

Estado	Significado
PROGRAMADA	Estado inicial al crear.
CONFIRMADA	Confirmación de la clínica.
ATENDIDA	Se asigna al registrar diagnóstico y tratamiento.
CANCELADA	Se asigna desde la ruta de cancelación.

Registrar atención a una cita cancelada devuelve HTTP 409. El cambio de estado solo admite PROGRAMADA, CONFIRMADA, ATENDIDA y CANCELADA.

Rutas principales

Servicio	Rutas
Categoría / Cliente / Producto / Pedido	/api/categorias, /api/clientes, /api/productos, /api/pedidos y rutas por ID.
Paciente	/api/pacientes y /api/pacientes/{id}.
Médico	/api/especialidades, /api/medicos, /api/horarios.
Citas	/api/citas, /api/citas/{id}/estado, /api/citas/{id}/cancelar.
Atención	/api/citas/{id}/atencion.

5. Preparación de las pruebas

En XAMPP, iniciar MySQL y ejecutar microservicios/database/create-databases.sql en phpMyAdmin. Abrir cada proyecto en VS Code y ejecutar ./mvnw.cmd spring-boot:run en una terminal independiente. Iniciar primero categoría, cliente, paciente y médico; después producto, pedido y citas.

La conexión predeterminada es localhost:3306, usuario root y contraseña vacía. Para cambiarla, configurar PROFILE_ACTIVE, SERVER_PORT, DB_URL, DB_USER y DB_PASSWORD.

Colección de Postman

Archivo microservicios/postman/Laboratorio-11.postman_collection.json. Incluye alta y validación de categoría/producto, cliente/pedido, paciente/especialidad/médico/horario/cita/atención, así como entradas inválidas e IDs que no existen.

Comprobación realizada

Los siete proyectos compilaron con Maven usando mvnw.cmd -q -DskipTests compile. No ejecuté solicitudes HTTP: la conexión local al puerto 3306 no respondió durante la preparación. No incluyo capturas falsas de Postman ni de aplicaciones en ejecución.

Evidencia visual

El diagrama anterior es un gráfico original de arquitectura. Para completar las evidencias de ejecución, se deben tomar capturas reales al correr Postman y XAMPP; así quedarán registradas las respuestas exitosas 200/201 y los escenarios inválidos 400/404/409.

6. Conclusiones

1. Al separar las funciones en microservicios entendí que cada servicio debe tener una responsabilidad específica y administrar sus datos.

2. Aprendí que los IDs que apuntan a otro microservicio se validan con una llamada HTTP, sin crear una clave foránea entre bases independientes.

3. Los perfiles me permiten cambiar puertos y conexiones según el ambiente sin modificar las clases Java.

4. Los DTO y Bean Validation ayudan a controlar las solicitudes y a responder con códigos HTTP que explican el resultado.

5. La compilación confirmó la estructura de los siete proyectos. Me falta iniciar MySQL de XAMPP y ejecutar Postman para comprobar los flujos en tiempo real.

Archivos de entrega

En Semana11/microservicios están los siete proyectos, el SQL de bases y la colección Postman. Este informe queda en Documentos/Laboratorios/Semana 11.